

SQL Practice Solutions Student & Hospital Domain

STUDENT DOMAIN SOLUTIONS

1. Retrieve all students

```
SELECT * FROM Students;
```

2. Show students from Mumbai

```
SELECT * FROM Students WHERE city='Mumbai';
```

3. Find students older than 21

```
SELECT * FROM Students WHERE age > 21;
```

4. Display all courses ordered by fees descending

```
SELECT * FROM Courses ORDER BY fees DESC;
```

5. Count total students department-wise

```
SELECT department, COUNT(*) FROM Students GROUP BY department;
```

6. Find average marks scored by students

```
SELECT AVG(marks) AS avg_marks FROM Enrollments;
```

7. Display highest marks in enrollments

```
SELECT MAX(marks) AS highest_marks FROM Enrollments;
```

8. Show students who enrolled in SQL course

```
SELECT s.student_name  
FROM Students s  
JOIN Enrollments e ON s.student_id=e.student_id  
JOIN Courses c ON e.course_id=c.course_id  
WHERE c.course_name='SQL';
```

9. Retrieve student names with course names using INNER JOIN

```
SELECT s.student_name, c.course_name  
FROM Students s  
JOIN Enrollments e ON s.student_id=e.student_id  
JOIN Courses c ON e.course_id=c.course_id;
```

10. Find students who did not enroll in any course

```
SELECT *  
FROM Students  
WHERE student_id NOT IN (  
SELECT student_id FROM Enrollments  
);
```

11. Show total enrollments per course

```
SELECT course_id, COUNT(*) AS total_enrollments  
FROM Enrollments  
GROUP BY course_id;
```

12. Display courses having more than 1 student enrolled

```
SELECT course_id, COUNT(*) AS total_students  
FROM Enrollments  
GROUP BY course_id  
HAVING COUNT(*) > 1;
```

13. Find students whose marks are greater than average marks

```
SELECT *  
FROM Enrollments  
WHERE marks > (  
SELECT AVG(marks) FROM Enrollments  
);
```

14. Retrieve top 3 students based on marks

```
SELECT *  
FROM Enrollments  
ORDER BY marks DESC  
LIMIT 3;
```

15. Show department-wise average marks

```
SELECT s.department, AVG(e.marks) AS avg_marks  
FROM Students s  
JOIN Enrollments e ON s.student_id=e.student_id  
GROUP BY s.department;
```

16. Find the student who paid the highest course fees

```
SELECT s.student_name, c.fees
FROM Students s
JOIN Enrollments e ON s.student_id=e.student_id
JOIN Courses c ON e.course_id=c.course_id
ORDER BY c.fees DESC
LIMIT 1;
```

17. Display courses with total revenue generated

```
SELECT c.course_name,
COUNT(e.student_id) * c.fees AS total_revenue
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id=e.course_id
GROUP BY c.course_name, c.fees;
```

18. Find duplicate cities in Students table

```
SELECT city, COUNT(*)
FROM Students
GROUP BY city
HAVING COUNT(*) > 1;
```

19. Retrieve students enrolled in more than one course

```
SELECT student_id, COUNT(*) AS total_courses
FROM Enrollments
GROUP BY student_id
HAVING COUNT(*) > 1;
```

20. Find the second highest marks scored by students

```
SELECT MAX(marks) AS second_highest_marks
FROM Enrollments
WHERE marks < (
SELECT MAX(marks) FROM Enrollments
);
```

HOSPITAL DOMAIN SOLUTIONS

1. Retrieve all patients

```
SELECT * FROM Patients;
```

2. Show patients from Mumbai

```
SELECT * FROM Patients WHERE city='Mumbai';
```

3. Find doctors specialized in Cardiology

```
SELECT * FROM Doctors WHERE specialization='Cardiology';
```

4. Display appointments ordered by bill amount descending

```
SELECT * FROM Appointments ORDER BY bill_amount DESC;
```

5. Count total patients city-wise

```
SELECT city, COUNT(*) FROM Patients GROUP BY city;
```

6. Find average bill amount

```
SELECT AVG(bill_amount) AS avg_bill FROM Appointments;
```

7. Display maximum bill amount

```
SELECT MAX(bill_amount) AS max_bill FROM Appointments;
```

8. Show patient names along with doctor names using JOIN

```
SELECT p.patient_name, d.doctor_name  
FROM Patients p  
JOIN Appointments a ON p.patient_id=a.patient_id  
JOIN Doctors d ON a.doctor_id=d.doctor_id;
```

9. Find patients who never booked an appointment

```
SELECT *  
FROM Patients  
WHERE patient_id NOT IN (  
SELECT patient_id FROM Appointments  
);
```

10. Display total appointments per doctor

```
SELECT doctor_id, COUNT(*) AS total_appointments  
FROM Appointments  
GROUP BY doctor_id;
```

11. Show doctors having more than 1 appointment

```
SELECT doctor_id, COUNT(*) AS total_appointments
FROM Appointments
GROUP BY doctor_id
HAVING COUNT(*) > 1;
```

12. Find patients whose bill amount is above average

```
SELECT *
FROM Appointments
WHERE bill_amount > (
SELECT AVG(bill_amount) FROM Appointments
);
```

13. Retrieve top 3 highest bills

```
SELECT *
FROM Appointments
ORDER BY bill_amount DESC
LIMIT 3;
```

14. Display specialization-wise total revenue

```
SELECT d.specialization,
SUM(a.bill_amount) AS total_revenue
FROM Doctors d
JOIN Appointments a ON d.doctor_id=a.doctor_id
GROUP BY d.specialization;
```

15. Find the doctor generating maximum revenue

```
SELECT d.doctor_name,
SUM(a.bill_amount) AS total_revenue
FROM Doctors d
JOIN Appointments a ON d.doctor_id=a.doctor_id
GROUP BY d.doctor_name
ORDER BY total_revenue DESC
LIMIT 1;
```

16. Show total revenue per month

```
SELECT MONTH(appointment_date) AS month,
SUM(bill_amount) AS total_revenue
FROM Appointments
GROUP BY MONTH(appointment_date);
```

17. Find duplicate cities in Patients table

```
SELECT city, COUNT(*)  
FROM Patients  
GROUP BY city  
HAVING COUNT(*) > 1;
```

18. Retrieve patients who visited more than once

```
SELECT patient_id, COUNT(*) AS visit_count  
FROM Appointments  
GROUP BY patient_id  
HAVING COUNT(*) > 1;
```

19. Find second highest bill amount

```
SELECT MAX(bill_amount) AS second_highest_bill  
FROM Appointments  
WHERE bill_amount < (  
SELECT MAX(bill_amount) FROM Appointments  
);
```

20. Display doctor-wise average bill amount

```
SELECT doctor_id,  
AVG(bill_amount) AS avg_bill  
FROM Appointments  
GROUP BY doctor_id;
```